

Object-oriented programming in C++

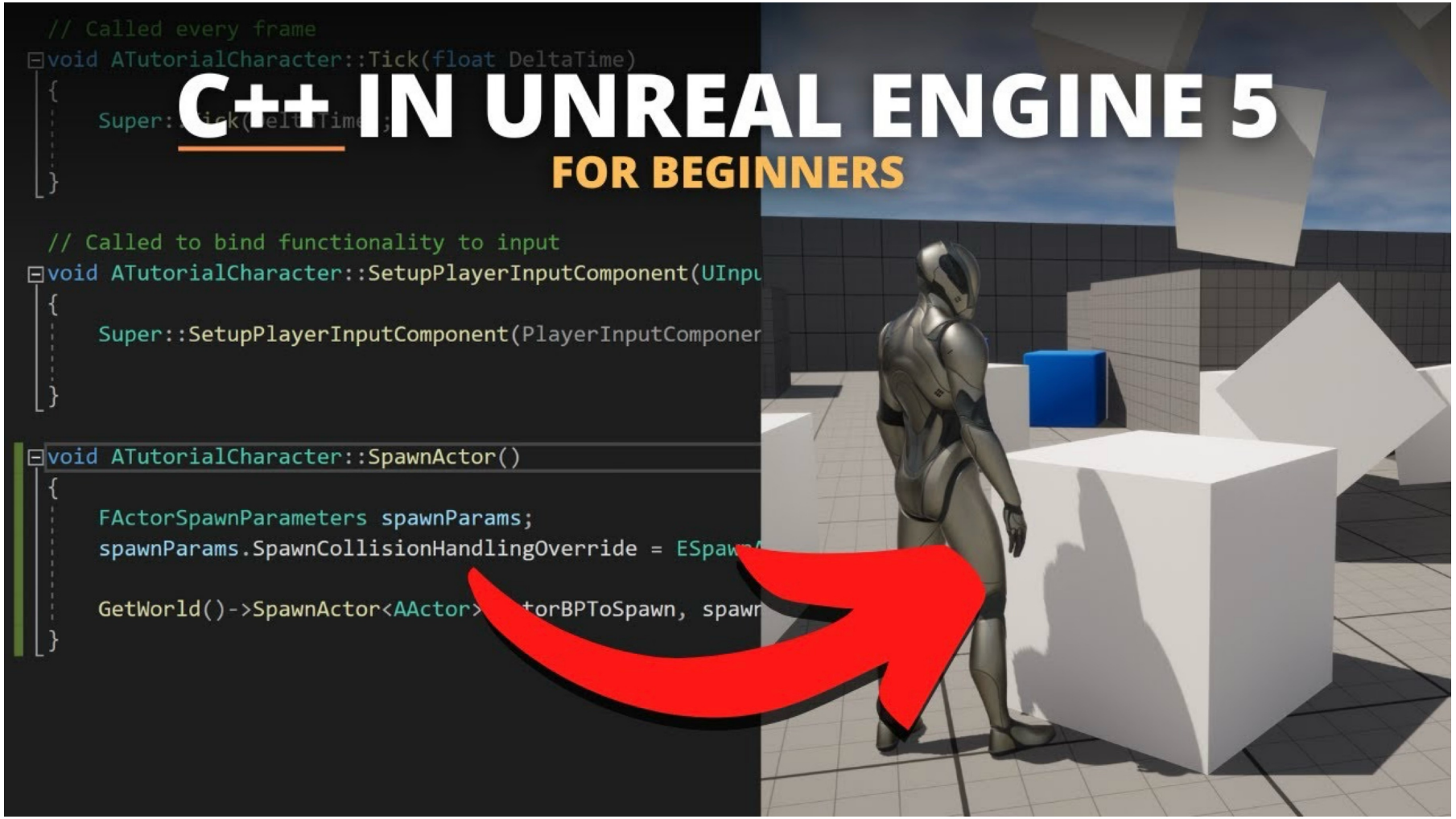
Introduction

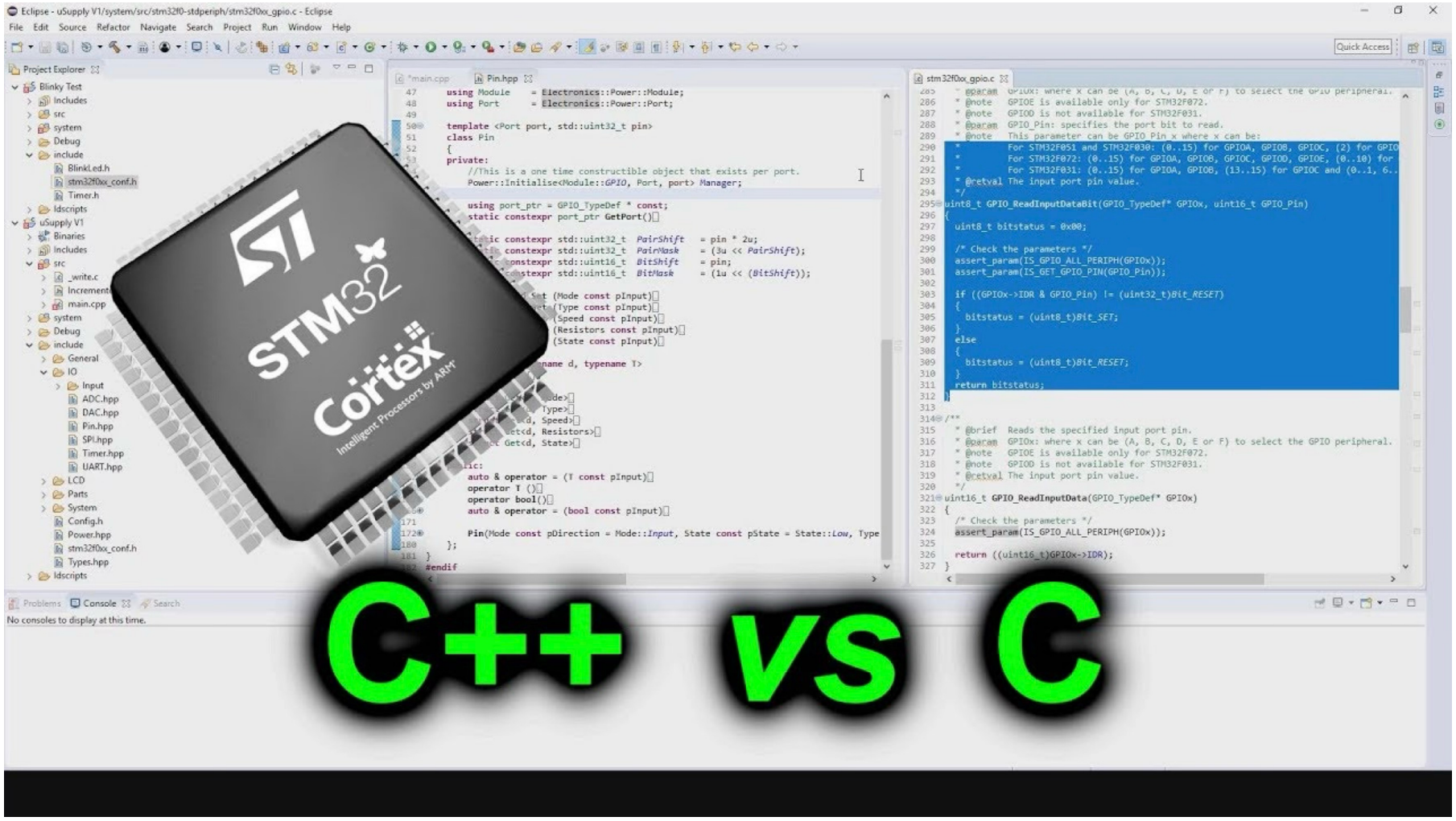
Dr. Claudia Weis, Faculty of Computer Science

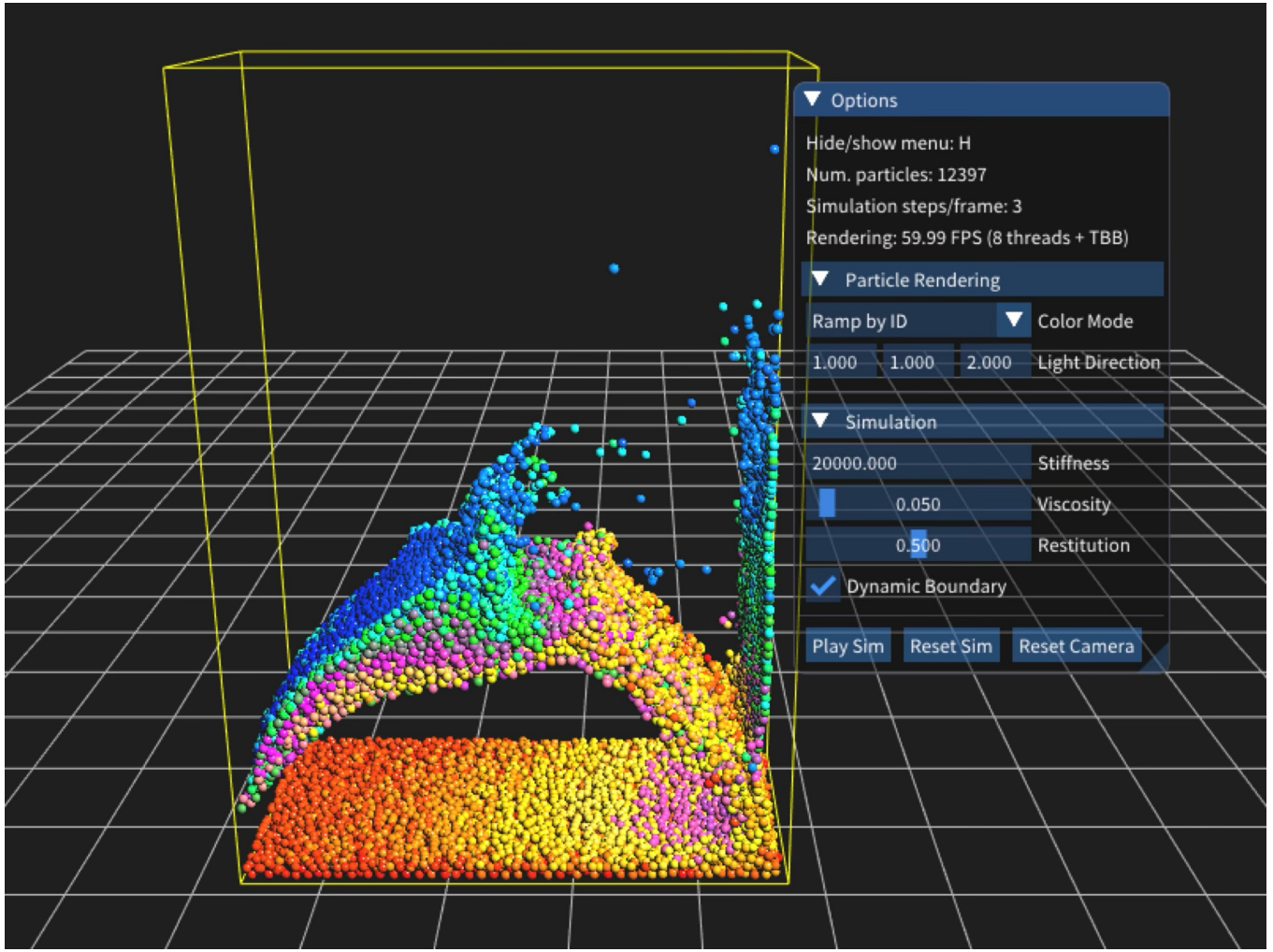
Fakultät für Informatik

UNIVERSITÄT DUISBURG ESSEN

Offen im Denken

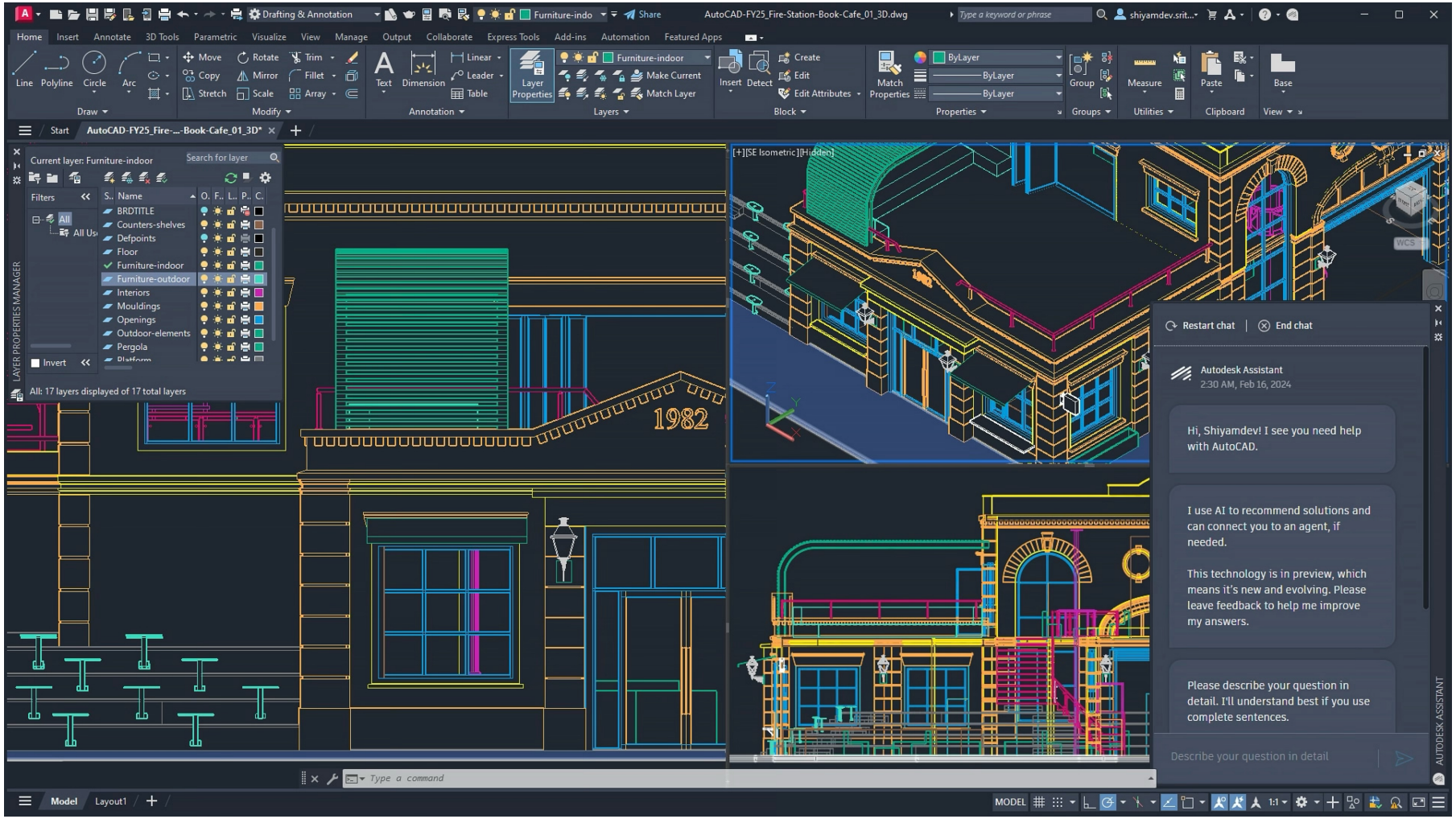

// Called every frame
void ATutorialCharacter::Tick(float DeltaTime)
{
}
// Called to bind functionality to input
void ATutorialCharacter::SetupPlayerInputComponent(UInpu
{
Super::SetupPlayerInputComponent(PlayerInputComponer
}
void ATutorialCharacter::SpawnActor()
{
FActorSpawnParameters spawnParams;
spawnParams.SpawnCollisionHandlingOverride = ESpaw
GetWorld()->SpawnActor<AActor>
torBPToSpawn, spawn
}
C++ IN UNREAL ENGINE 5
FOR BEGINNERS


C++ vs C


Options
Hide/show menu: H
Num. particles: 12397
Simulation steps/frame: 3
Rendering: 59.99 FPS (8 threads + TBB)
Particle Rendering
Ramp by ID
Color Mode
1.000
1.000
2.000
Light Direction
Simulation
20000.000
Stiffness
0.050
Viscosity
0.500
Restitution
Dynamic Boundary
Play Sim
Reset Sim
Reset Camera


PRO EDU
LEVELS
PHOTOSHOP TOOLS & TERMINOLOGY
Levels
Preset: Default
Channel: RGB
Input Levels:
Output Levels:
OK
Cancel
Auto
Options...
Preview
Ps

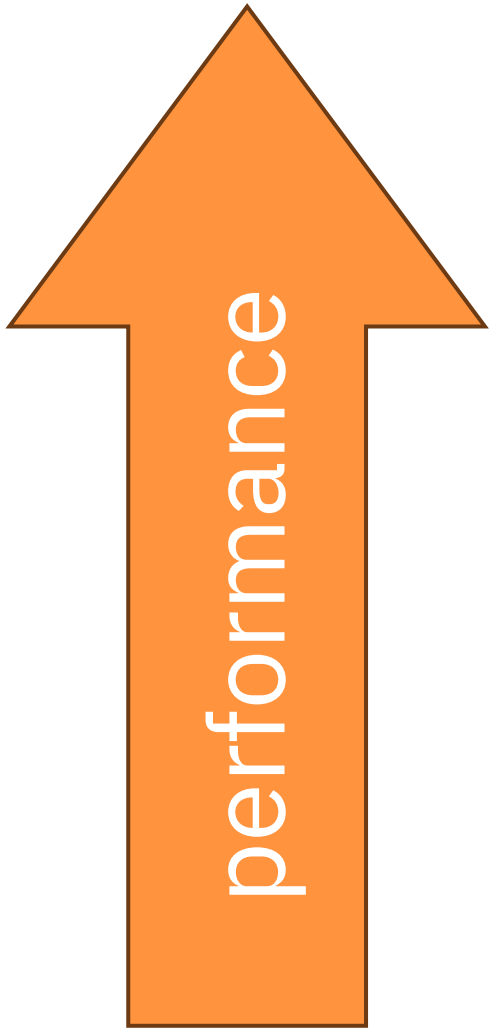

Windows 11


Drafting & Annotation
AutoCAD-FY25_Fire-Station-Book-Cafe_01_3D.dwg
Home
Insert
Annotate
3D Tools
Parametric
Visualize
View
Manage
Output
Collaborate
Express Tools
Add-ins
Automation
Featured Apps
Line
Polyline
Circle
Arc
Move
Copy
Stretch
Rotate
Mirror
Scale
Trim
Fillet
Array
Text
Dimension
Linear
Leader
Table
Layer Properties
Make Current
Match Layer
Insert
Detect
Create
Edit
Edit Attributes
Match Properties
ByLayer
Group
Measure
Paste
Base
Draw
Modify
Annotation
Layers
Block
Properties
Groups
Utilities
Clipboard
View
Start
AutoCAD-FY25_Fire-...-Book-Cafe_01_3D*
Current layer: Furniture-indoor
Search for layer
Filters
Name
BRDTITLE
Counters-shelves
Defpoints
Floor
Furniture-indoor
Furniture-outdoor
Interiors
Mouldings
Openings
Outdoor-elements
Pergola
Invert
All: 17 layers displayed of 17 total layers
LAYER PROPERTIES MANAGER
[+][SE Isometric][Hidden]
1982
Restart chat
End chat
Autodesk Assistant
2:30 AM, Feb 16, 2024
Hi, Shiyamdev! I see you need help with AutoCAD.
I use AI to recommend solutions and can connect you to an agent, if needed.
This technology is in preview, which means it's new and evolving. Please leave feedback to help me improve my answers.
Please describe your question in detail. I'll understand best if you use complete sentences.
Describe your question in detail
AUTODESK ASSISTANT
Type a command
Model
Layout1
MODEL

C++ is powerful.

C++ is THE language if you want to write efficient software.

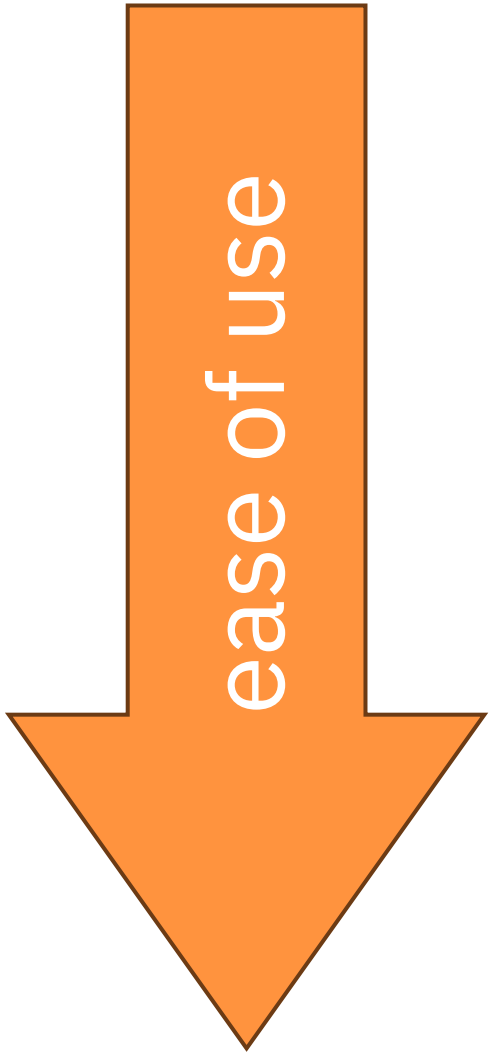
User-friendliness vs. performance



C++

Java

Python



C++ is very hard to learn, extremely hard to master.

Luckily, you don’t need to be experts at the end of the course!

What we learn (hopefully)

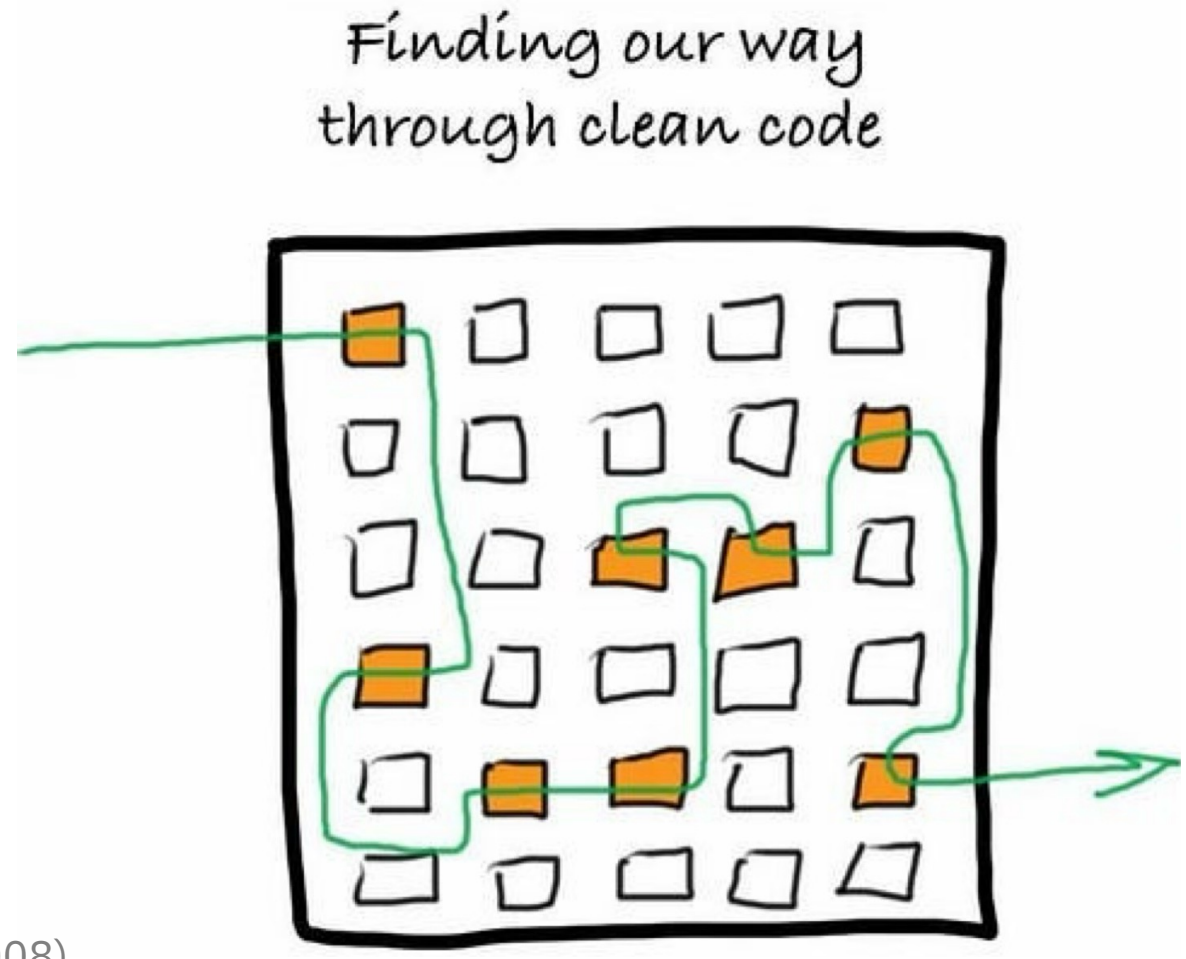
- Deconstruct a problem into smaller chunks
- Use object-orientation to model problems
- Structure functionality
- Understand how a computer stores/computes data
- Generalize and abstract concepts
- We don’t learn C++ (strictly speaking), we learn the above … using C++

Programming is the ability to explain a problem in all details



Source: Visualization based on Robert C. Martin (*Clean Code*, 2008) / Community illustration

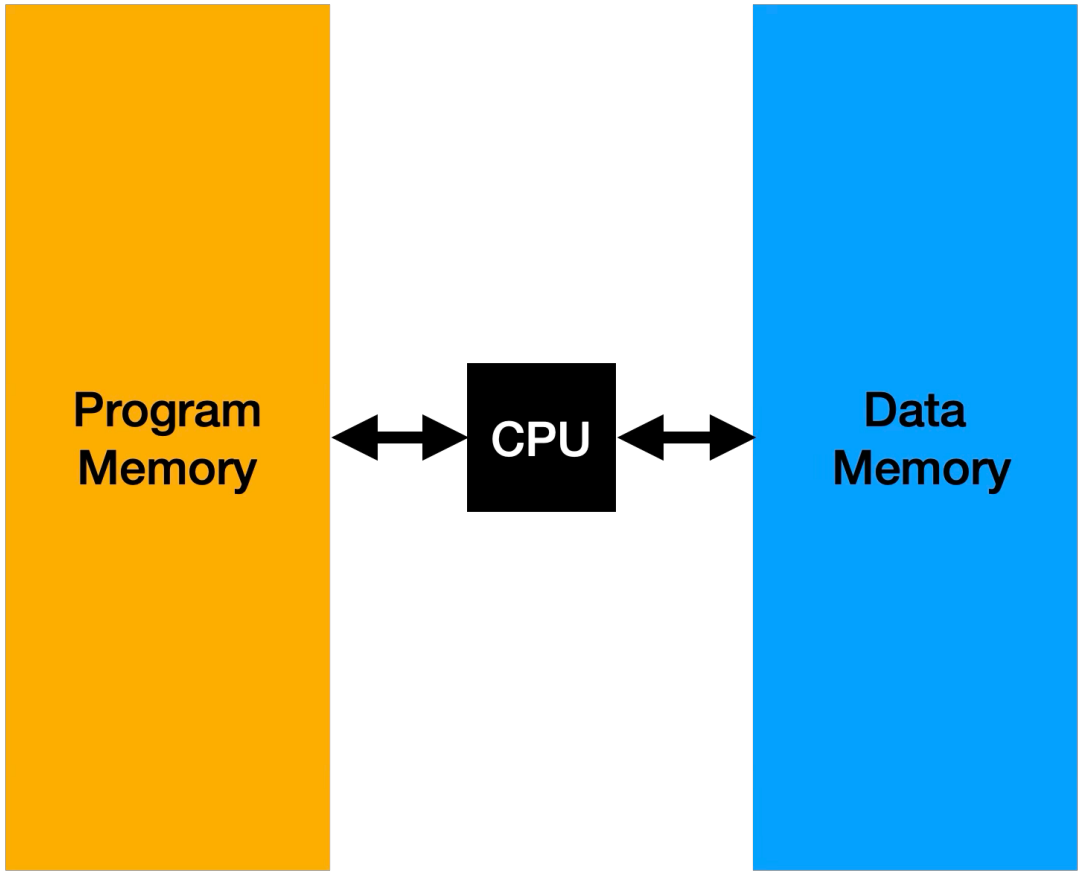
Programming is the art to explain a problem such that someone else understands it



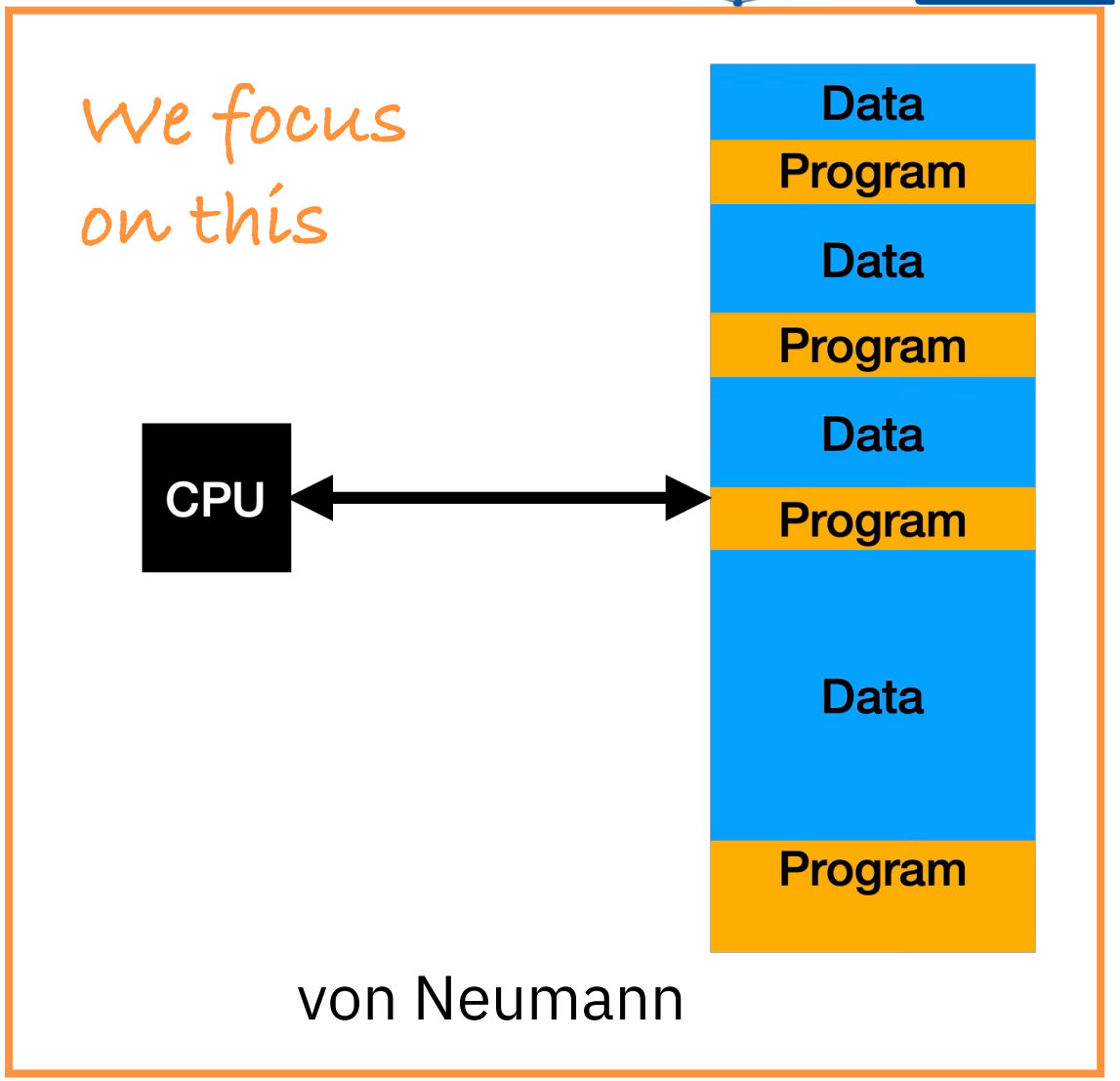
Source: Visualization based on Robert C. Martin (*Clean Code*, 2008) / Community illustration

To be good programmers,
we need to understand our computer first!

Computer architectures: Harvard vs. von Neumann



Harvard

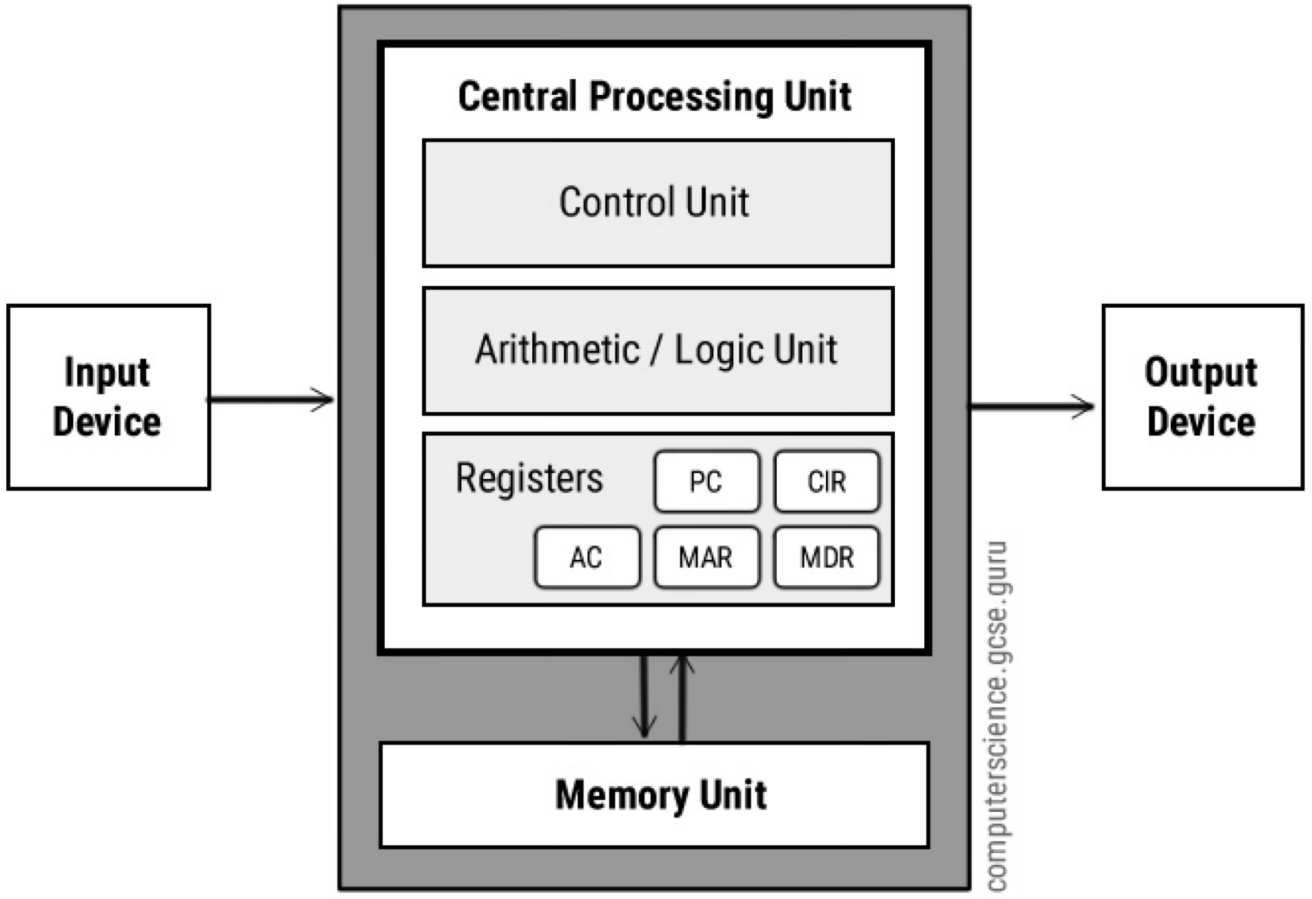


von Neumann

Von Neumann Architecture (most PCs work like this)

- Memory
  (RAM)
- CPU
  (Intel/AMD/Apple/others)
- Bus
  (connects components)
- I/O devices
  (GPU, Hard drive, network interface)

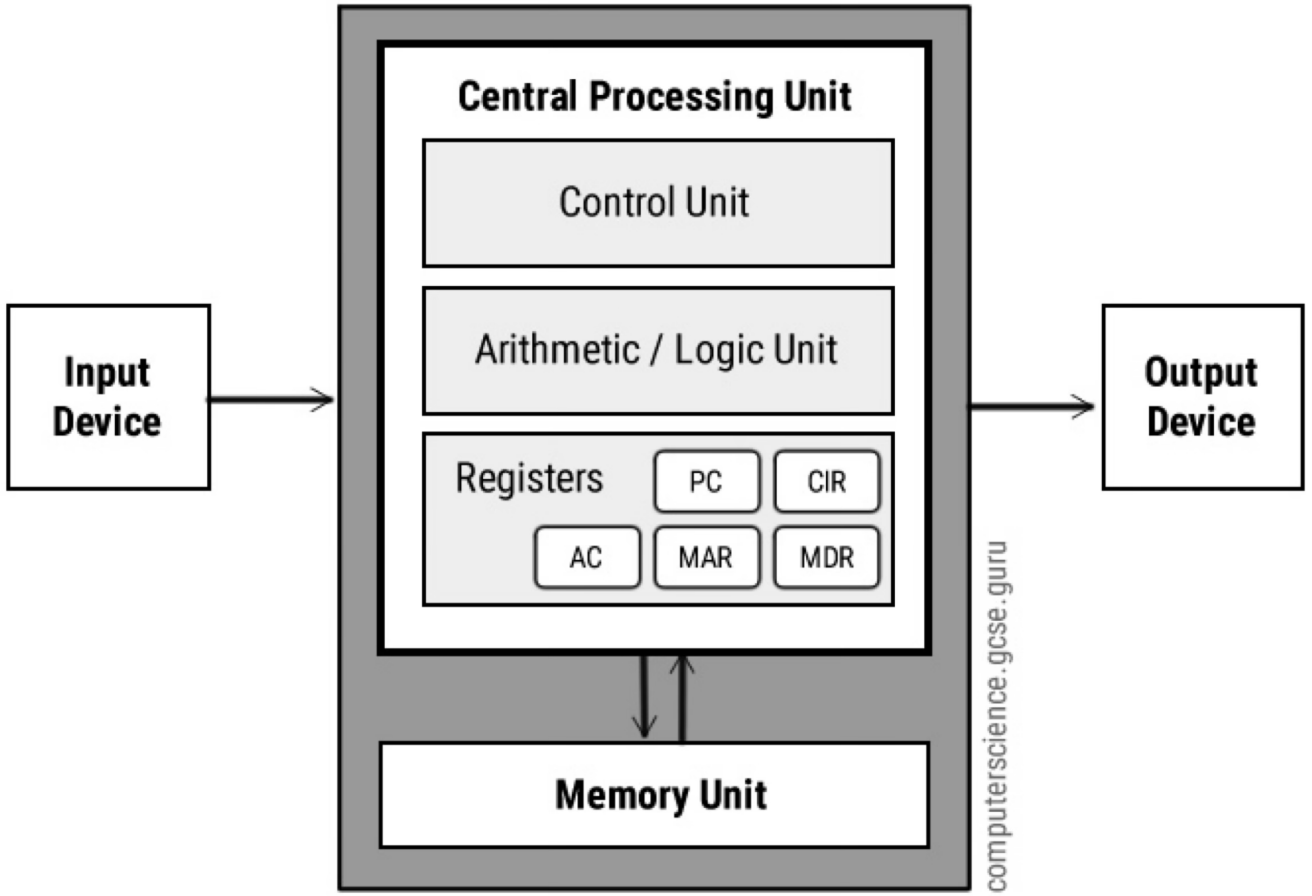
- **Programmable** through software that you write



Memory and instructions

- Addressable memory cells (RAM)
- Programs can access the memory (more or less) freely
- A program consists of instructions that are also stored in memory
- Instructions are loaded by the CPU, and executed

- Data can be stored/read in memory through instructions
  - „read(address), write(address, value)“



Execution in the von Neumann Model

1. Load command from memory
2. Execute it (e.g. add two data items)
3. Load next command from memory
4. Execute it
5. Load next command from memory
6. Execute it

Instruction order

- Address of next command is stored in the instruction counter register
- (Conditional) jumps change what the next command is, i.e. allow control flow

Condition is true

```
int number = 5;

if (number > 0) {
  // code
}
else {
  // code
}

// code after if...else
```

Condition is false

```
int number = 5;

if (number < 0) {
  // code
}
else {
  // code
}

// code after if...else
```

Von Neumann languages

Programming languages that use the Von Neumann model

- ✦ Named variables as handles to specific memory cells
- ✦ Control flow with jumps (if/else) or control structures (while/for)

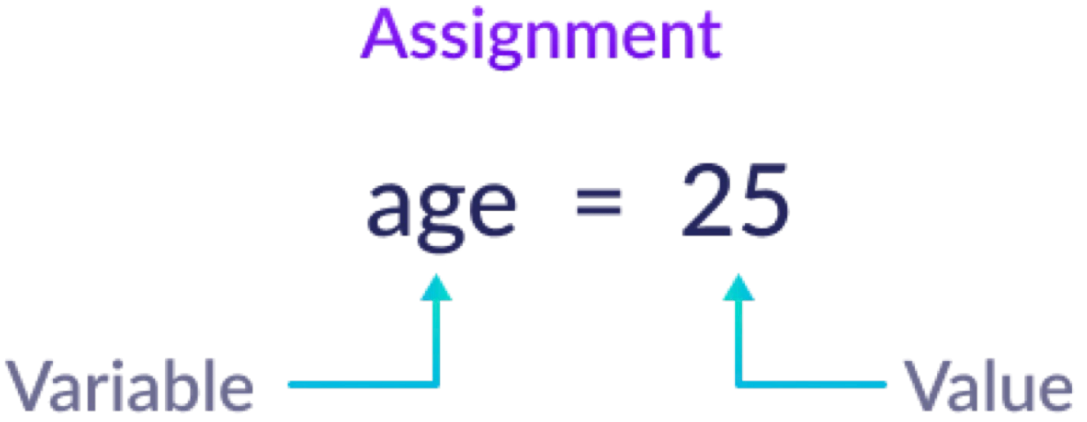
- ➢ Set values in memory cells
- ➢ Set in which order stuff happens

Summary

- CPU, Memory, Bus, Input/Output devices
- Instruction: Address & Operation
- CPU fetches next address, executes operation
- Variables are used to refer to memory at certain locations

Variables and assignments
x = x+1 (Mathematicians hate this trick)

- Easy to make mistakes, if we have direct access
- Managing memory cells difficult?
  - Introduce the concept of variables
- Address, size/type, content, mutability, lifetime



- Better, but humans still make errors

Control structures

- Control structures and functions specify sequences of program execution
- Operators change the content of memory cells

Example

bla.cpp

```cpp
#include <iostream>

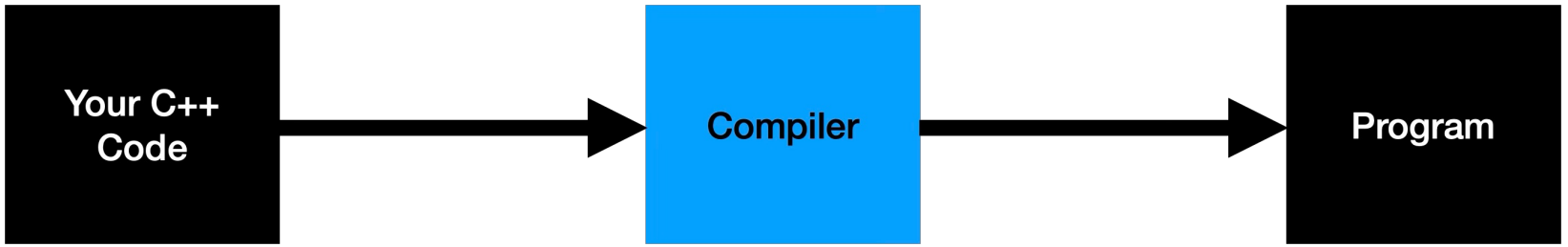
int main()
{
    int x = 7;
    while (x > 2)
    {
        if (x % 2 == 0)
        {
            x = x / 2;
        }
        else
        {
            x = x * 3 + 1;
        }
        std::cout << x << std::endl;
    }
}
```

```
❯ ./a.out
22
11
34
17
52
26
13
40
20
10
5
16
8
4
2
```

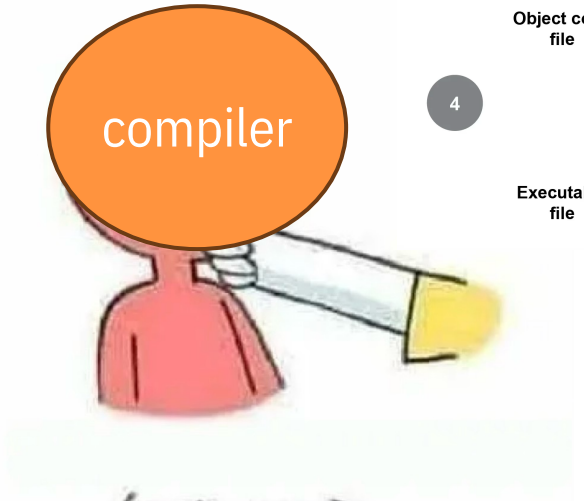
Compiling: from source code to an executable programme

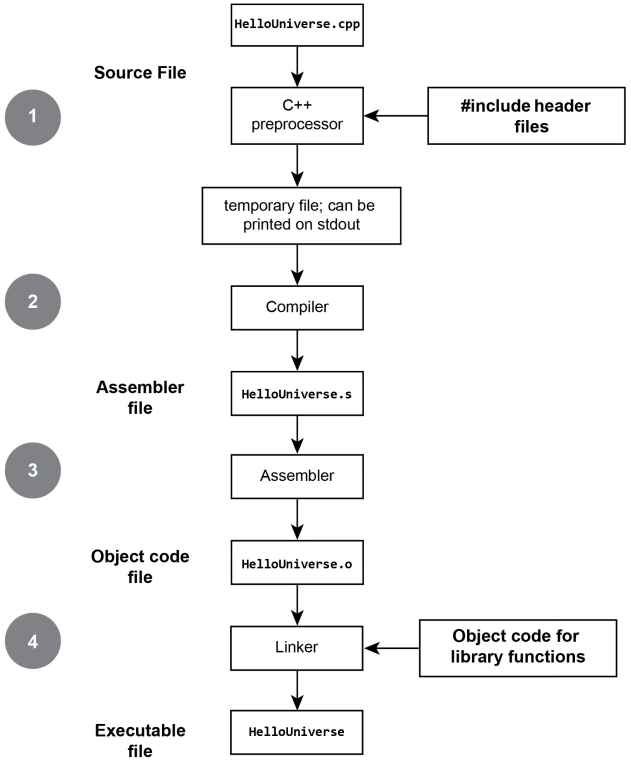
- C++ is a compiled language.
- You write source code.
- The compiler takes your code and transforms it into an executable program.

- Many intermediate steps
- e.g. link external library for GUI




HEY Compiler WHY DO YOU ALWAYS WEAR THAT MASK?
compiler


compiler


HelloUniverse.cpp
Source File
1
C++ preprocessor
#include header files
temporary file; can be printed on stdout
2
Compiler
Assembler file
HelloUniverse.s
3
Assembler
Object code file
HelloUniverse.o
4
Linker
Object code for library functions
Executable file
HelloUniverse


LET'S KEEP THIS ON.
compiler
OWLTURD.COM

Runtime
What is there, even if you don't think about it

- Your code rarely runs in isolation.
- Runtime cares for initialisation and clean-up
- Error-handling
- Runtime checks
- Dynamic memory management

➢ Short take-home message: Stuff happens even if you don't write it

No more details?
We'll get into more details if we need to

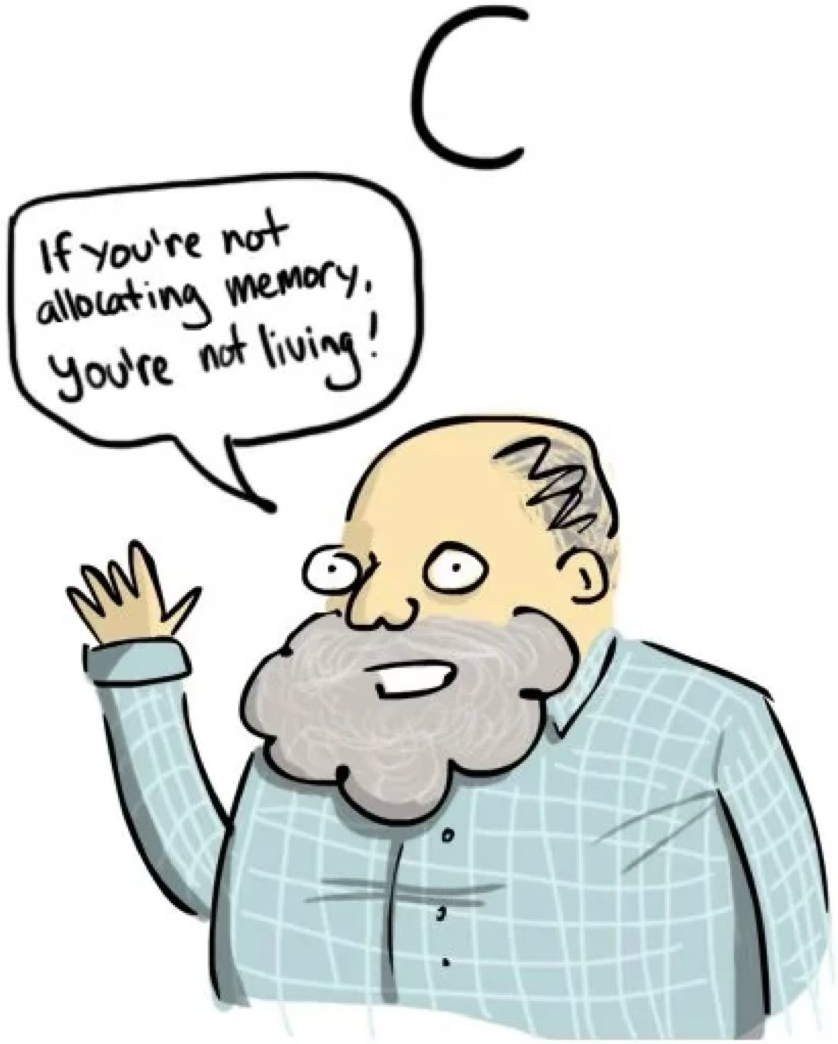
- Unless you want to build a compiler yourself…
- You (most likely) don't need to get into the details of most things
- You probably will need to adapt some existing C++ code or write simple software

➢ Aim: Teach as much C++ as possible, while sparing as many details as possible

Fakultät für Informatik

History of C++

Assuming you know a bit of C



https://leftoversalad.com/c/015_programmingpeople/

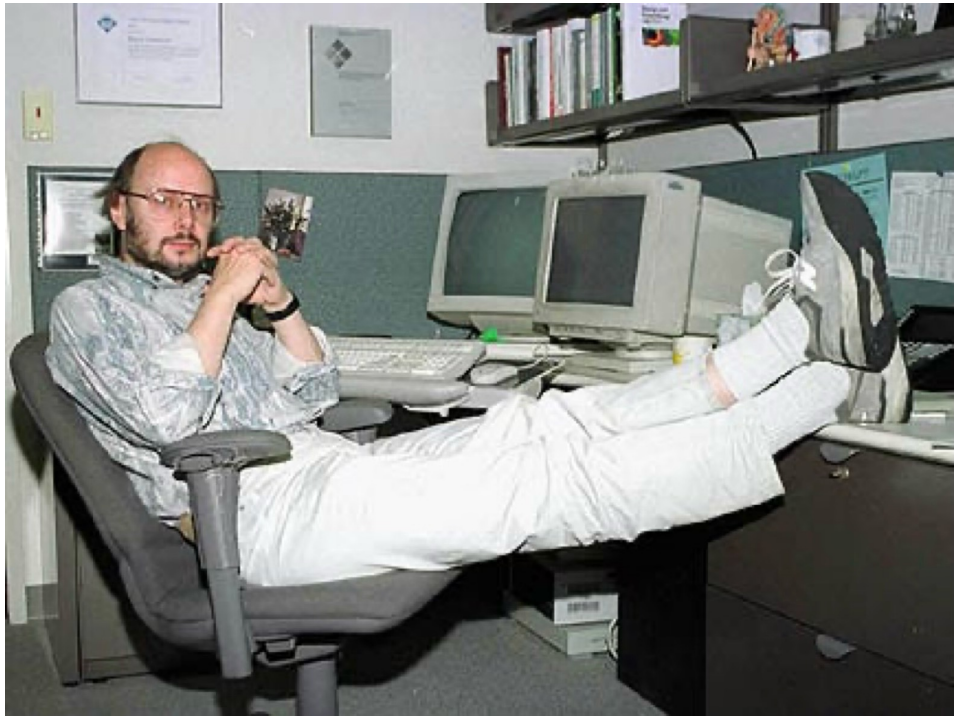
Let's look a bit at the history of C++



https://leftoversalad.com/c/015_programmingpeople/

“Simula is cool but slow“
-Bjarne Stroustroup, probably

- During his PhD thesis, Bjarne worked with Simula
- Simula introduced object-orientation, classes and much more that is used to this day
- Simula was quite slow (it’s 1979, computers are nowhere near todays performance)
- Bjarne enjoyed these features so much, he began working on something new



Popular photo of Stroustroup., taken in his office at the AT&T Bell Laboratories in Murray Hill, New Jersey, approx. 1990

“C with classes”

- Enhance C with some Simula-like features
- C was chosen because it was high-level (at the time), widely used and fast

- 1982 first „C++“ compiler Cfront
- 1985 release of C++ 1.0
- 1989 C++ 2.0 with major improvements
- 1998 release of C++98 (first real standard)

Modern C++
A lot has changed

- C++11 is the first „modern“ C++ version
- C++14, C++17, C++20 add tons of new features

- We will not go through all cool keywords and details of things you can do now

- Challenge: all old features are still available and used throughout codebases
- Many concurrent approaches to do a thing

- The internet does not forget.
- You will find old-fashioned solutions, which in modern times are considered bad.

Example: Getting user input

```
#include <iostream>
#include <string>

  int main()
  {
      std::string mystr;

      std::cout << "What is your name?" << std::endl; // print to command line

      std::getline(std::cin, mystr); // read from command line

      std::cout << "Welcome " << mystr << ".\n";
      return 0;
  }
```

Getting user input - 10 different options
Not all are good

safe (and modern) methods

- using C++-Streams
  (with std::string and safe handling)
  - `std::getline(std::cin, str);`
  - `std::cin.get(char_variable);`
  - `std::getline(std::ifstream, str);`

relatively safe methods

- using C strings with size limit
  - `cin.getline(buffer, size);`
  - `scanf_s("%s", buffer, size);`

unsafe methods

- mostly old methods for C strings, some without size limit
  - `cin >> str;`
  - `fgets(buffer, size, stdin);`
  - `scanf("%s", buffer);`
  - `gets(buffer);`
  - `sscanf(buffer, "%s", str);`

Key question: What can go wrong?



Here: A buffer overflow can leak into neighbouring memory and cause unwanted effects.

Another example: Rule of 0/3/5/6/6.5

- Rule of Three: If a class defines one of destructor, copy constructor, or copy assignment, it should define all three.
- Rule of Five: If a class defines any of the above three or the move constructor/assignment operator, it should define all five.
- Rule of Zero: If a class does not manage resources, rely on the compiler-generated defaults.
- Rule of Six: If a class defines all five special functions, consider defining a swap function as well.
- Rule of Six-and-a-Half: If using the class in hashed containers, add a hash function.

Rule of 0/3/5/6/6.5 – What are they good for?

- established best practices and design guidelines in C++
  - how to handle so-called Special Member Functions (such as destructors and constructors)
  - proper resource management (e.g., managing memory, file handles, or network connections).

- guide for writing clean, safe, and performant C++
- prevent your program from crashing, leaking memory, or behaving unpredictably during copies and assignments

What to do?
Good question ;-)

- It can become quite complicated to deal with this
- C++ is old, it needs much backwards compatibility for legacy software
- New features make old features obsolete at times

- Best practise: Learn the modern way, try to learn old ways if you encounter them
- That's what we'll try here

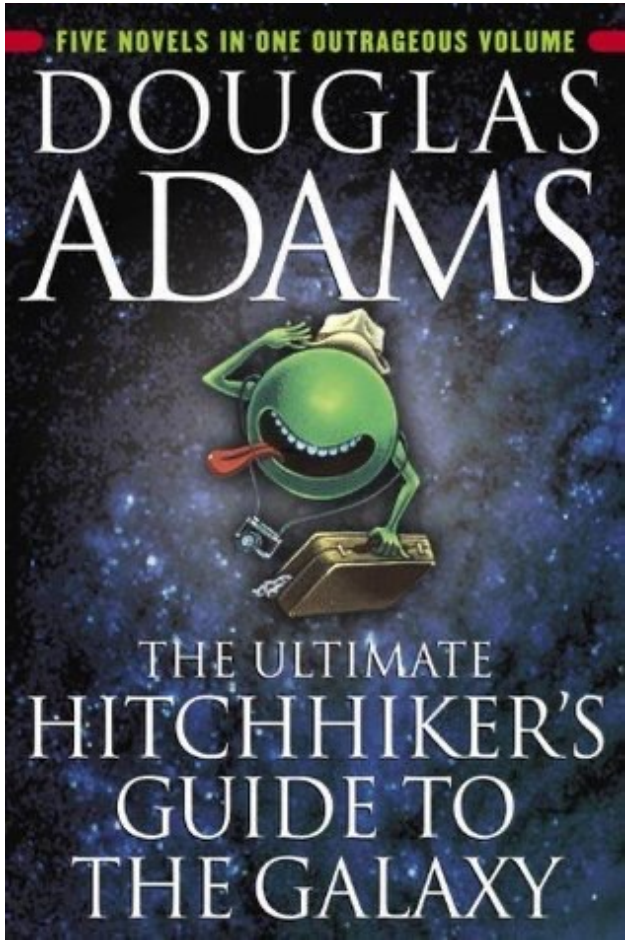


And for the exam?

Fakultät für Informatik

UDE

Don’t panic!



- Learning is not only for the exam.
- General aim: make you find your strategies to deal with the overwhelming masses of “C++ stuff”
- Special objective: Teach you useful pieces and examples to implement personal smaller tasks and make it through the exam.

Until next week – a little homework

- Install CLion (it is cross-platform and free of charge for non-commercial use)
  - It is okay to use other IDEs at your own responsibility.

- Work on exercise sheet #1.
- Objective: obtain a programming environment for C++ that works on your own computer.

- Feel free to bring your devices to the exercise session.

Fakultät für Informatik

Questions?



"live" in lecture and exercise



discussion forum @ Moodle (preferred, everyone can reply)



by e-mail (use your @uni-due.de email address!)